

Part A: Theory Questions

1. What is representation learning? Explain its objective and applications.

Representation learning, also known as feature learning, is a type of machine learning in which the system automatically discovers useful features or representations from raw data using neural networks. It is mainly used when data is unstructured or unlabeled.

Objective:

The main objective of representation learning is to find an appropriate representation of data based on features in order to perform machine learning tasks efficiently. It reduces the need for manual feature engineering.

Working:

- Input data is provided to the neural network.
- Hidden layers automatically extract important features.
- The learned features are used for classification or prediction.

Applications:

1. Speech recognition
2. Image recognition
3. Signal processing
4. Natural language processing
5. Music recognition

Conclusion:

Representation learning enables machines to automatically learn useful patterns from data and improves performance in complex machine learning tasks.

2. Explain the concept of Independent Component Analysis (ICA). How is it different from PCA?

Independent Component Analysis (ICA) is an unsupervised learning technique used to separate independent source signals from observed mixed signals.

The ICA model is represented as:

$$X = AS$$

where:

- X = observed mixed data
- A = mixing matrix
- S = independent source signals

The objective is to estimate an unmixing matrix W such that:

$$U = WX$$

where U approximates the original independent signals.

Difference between ICA and PCA

ICA	PCA
Components are statistically independent	Components are only uncorrelated
Uses higher-order statistics	Uses variance and covariance
Suitable for signal separation	Suitable for dimensionality reduction
More powerful for complex data	Simpler mathematical approach

Applications:

- Audio signal separation
- EEG signal analysis
- Image processing

Conclusion:

ICA is more powerful than PCA for separating hidden independent signals from mixed observations.

3. What are autoencoders? Describe their working and objective function.

Autoencoders are neural networks used for representation learning and dimensionality reduction. They attempt to reproduce the input at the output layer.

Structure:

An autoencoder contains:

1. Input layer
2. Hidden layer (encoder)
3. Output layer (decoder)

Working:

- Encoder converts input into compressed representation.
- Decoder reconstructs original input from compressed representation.

The process is:

$$x' = \text{Decode}(\text{Encode}(x))$$

Mathematical Representation:

Hidden layer:

$$Z = f(Wx)$$

Output layer:

$$Y = g(VZ)$$

Objective Function:

$$J = \sum [X_n - VWx^{(n)}]^2$$

Applications:

- Data compression
- Noise reduction
- Feature extraction
- Image reconstruction

Conclusion:

Autoencoders learn compact representations of data while preserving important information.

4. Define active learning. Explain its query strategies with examples.

Active learning is a semi-supervised learning method in which the learning algorithm interactively queries the user to obtain labels for selected data points.

Working:

- Begin with small labeled data.
- Train model.
- Select most informative unlabeled samples.
- Obtain labels and retrain model.

Query Strategies:

1. **Uncertainty Sampling**
Select data points where model confidence is minimum.
2. **Query by Committee**
Multiple models vote; samples with highest disagreement are selected.
3. **Expected Model Change**
Select samples that produce maximum change in model.
4. **Expected Error Reduction**
Select samples reducing future prediction error.
5. **Variance Reduction**
Select samples reducing output variance.

Example:

In medical diagnosis, the model asks experts to label uncertain patient records.

Conclusion:

Active learning minimizes labeling cost while improving model performance.

5. What is instance-based learning? Discuss its characteristics and give examples.

Instance-based learning is a memory-based learning approach in which new instances are compared with previously stored training examples rather than creating an explicit generalized model.

Characteristics:

1. Stores training instances directly in memory
2. Learning occurs during prediction time
3. Computational complexity increases with number of instances
4. Uses similarity measures such as Euclidean distance

Examples:

- K-Nearest Neighbour (K-NN)
- Support Vector Machine (SVM)
- PCA
- Radial Basis Function (RBF)

Conclusion:

Instance-based learning is effective for similarity-based classification problems.

6. Explain ensemble learning and describe any two ensemble techniques such as bagging and boosting.

Ensemble learning combines multiple weak learning models to produce a stronger prediction model.

Advantages:

- Higher accuracy
- Reduced overfitting
- Better generalization

(a) Bagging

Bagging stands for Bootstrap Aggregation. It uses bootstrap sampling with replacement.

Steps:

1. Create bootstrap samples
2. Train multiple classifiers
3. Perform majority voting

Example:

Random Forest algorithm.

(b) Boosting

Boosting is an iterative technique where incorrectly classified samples receive higher weights.

Steps:

1. Train weak learners sequentially
2. Increase weights of misclassified data
3. Combine weak learners

Example:

AdaBoost.

Conclusion:

Ensemble learning improves prediction performance by combining multiple learners.

7. What is Reinforcement Learning? Explain its key characteristics and give real-world examples.

Reinforcement Learning (RL) is a machine learning approach in which an agent learns by interacting with an environment and receiving rewards or penalties.

Key Characteristics:

1. Trial-and-error learning
2. Uses rewards and punishments
3. Learns optimal policy
4. Balances exploration and exploitation
5. Maximizes cumulative reward

Components:

- Agent
- Environment
- State
- Action
- Reward
- Policy

Examples:

- Self-driving cars
- Chess playing systems
- Robot navigation

Conclusion:

RL enables intelligent agents to learn optimal actions through interaction.

8. What is a policy in Reinforcement Learning? Differentiate between deterministic and stochastic policies.

A policy is a strategy used by the agent to decide actions based on current states.

Deterministic policy:

$$\pi(s) \rightarrow a$$

Stochastic policy:

$$\pi(s) \rightarrow p(a)$$

Difference:

Deterministic Policy	Stochastic Policy
Returns one action	Returns probability distribution
Fixed action for state	Multiple possible actions
Simple	More flexible

Conclusion:

Policies guide the decision-making process of reinforcement learning agents.

9. Explain exploration vs exploitation with suitable examples.

Exploration means trying new actions to discover better rewards, whereas exploitation means selecting known actions with highest reward.

Exploration:

- Tries unknown paths
- Gains new knowledge

Exploitation:

- Uses known best action
- Maximizes immediate reward

Example:

A food delivery robot:

- Exploring → trying new route
- Exploiting → using shortest known route

Conclusion:

A balance between exploration and exploitation is essential for optimal learning.

10. What is the Markov Property? Why is it important in Reinforcement Learning?

The Markov Property states that the next state depends only on the current state and action, not on past history.

Mathematically:

$$P(S_{t+1} | S_t) = P(S_{t+1} | S_1, S_2, \dots, S_t)$$

Importance:

1. Simplifies decision-making
2. Forms basis of Markov Decision Process (MDP)
3. Enables efficient computation
4. Used in dynamic programming and Q-learning

Conclusion:

The Markov Property is fundamental for modeling sequential decision-making problems.

11. Define a Markov Chain. Explain transition probability with an example.

A Markov Chain is a stochastic process satisfying the Markov Property.

Transition Probability:

It represents probability of moving from one state to another.

Example:

Weather prediction:

Current State Sunny Rainy

Sunny	0.7	0.3
Rainy	0.4	0.6

This means:

- Probability of sunny after sunny = 0.7
- Probability of rainy after rainy = 0.6

Conclusion:

Markov Chains model systems where future states depend only on present state.

12. What is a Markov Reward Process (MRP)? Explain its components.

A Markov Reward Process is an extension of a Markov Chain that includes rewards.

It is represented as:

$$(S, P, R, \gamma)$$

Components:

1. $S \rightarrow$ set of states
2. $P \rightarrow$ transition probability matrix
3. $R \rightarrow$ reward function
4. $\gamma \rightarrow$ discount factor

Purpose:

Evaluates long-term usefulness of states.

Conclusion:

MRP combines state transitions and rewards for value estimation.

13. What is a value function? Explain the difference between reward and return.

A value function measures long-term cumulative reward obtained from a state.

Reward:

- Immediate feedback
- Short-term gain

Return:

- Total accumulated reward over time

Return equation:

$$G_t = \sum_{n=0}^{\infty} \gamma^n R_{t+n+1}$$

Difference:

Reward	Return
Immediate	Long-term
Single step	Sum of rewards
Short-term objective	Overall objective

Conclusion:

Reward evaluates immediate action while return evaluates long-term success.

14. Explain the Bellman Equation and its significance in Reinforcement Learning.

The Bellman Equation expresses value of a state recursively as immediate reward plus discounted future rewards.

Bellman Equation:

$$v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s', r} P(s', r | s, a) [r + \gamma v_{\pi}(s')]$$

Significance:

1. Basis of Dynamic Programming
2. Used in Value Iteration
3. Helps find optimal policy
4. Computes optimal value function

Conclusion:

Bellman Equation is the foundation of reinforcement learning algorithms.

15. Differentiate between:

(a) Dynamic Programming

(b) Monte Carlo Methods

(c) Temporal Difference Learning

Dynamic Programming	Monte Carlo	Temporal Difference
Requires complete environment model	Model-free	Model-free
Uses Bellman equation	Uses episode returns	Uses bootstrapping
Updates using expected values	Updates after episode completion	Updates after each step
Computationally expensive	Simple	Efficient
Example: Value Iteration	Example: MC Prediction	Example: SARSA, Q-learning

Conclusion:

These methods differ mainly in how they estimate value functions and learn from experience.

Part B: Lab Questions

1. ICA Implementation

```
import numpy as np
from sklearn.decomposition import FastICA

# Source signals
S = np.array([
    [1, 1],
    [2, -1],
    [3, 1],
    [4, -1]
])

# Mixing matrix
A = np.array([
    [1, 2],
    [3, 4]
])

# Mixed signals
X = S @ A.T

# ICA
ica = FastICA(n_components=2, random_state=0)
U = ica.fit_transform(X)

print("Original Signals:\n", S)
print("Recovered Signals:\n", U)
```

2. Autoencoder

```
import numpy as np
from sklearn.neural_network import MLPRegressor

X = np.array([
    [1, 2, 3, 4, 5],
    [2, 3, 4, 5, 6],
    [3, 4, 5, 6, 7],
    [4, 5, 6, 7, 8]
], dtype=float)

autoencoder = MLPRegressor(
    hidden_layer_sizes=(2,),
    activation="relu",
    max_iter=5000,
    random_state=0
)

autoencoder.fit(X, X)
```

```
X_reconstructed = autoencoder.predict(X)

W = autoencoder.coefs_[0]
b = autoencoder.intercepts_[0]

hidden = np.maximum(0, X @ W + b)

print("Encoded Representation:\n", hidden)
print("Reconstructed Output:\n", X_reconstructed)
```

3. Active Learning

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression

X, y = make_classification(
    n_samples=120,
    n_features=2,
    n_redundant=0,
    n_informative=2,
    random_state=0
)

labeled = np.arange(10)
unlabeled = np.arange(10, 120)

model = LogisticRegression(solver="liblinear")

for i in range(8):

    model.fit(X[labeled], y[labeled])

    probs = model.predict_proba(X[unlabeled])[:,1]

    query = np.argmin(np.abs(probs - 0.5))

    labeled = np.append(labeled, unlabeled[query])

    unlabeled = np.delete(unlabeled, query)

    print("Iteration:", i+1,
          "| Labeled Samples:", len(labeled))
```

4. Radial Basis Function (RBF)

```
import numpy as np

X = np.array([1,3,5])
centers = np.array([2,4])

sigma = 1

for x in X:
    for c in centers:

        distance = np.linalg.norm(x-c)
```

```

rbf = np.exp(-(distance**2)/(2*sigma**2))

print("Input:", x,
      "Center:", c,
      "Distance:", distance,
      "RBF:", round(rbf,4))

```

5. Apriori Algorithm

```

transactions = [
    {"A", "B"},
    {"A", "C"},
    {"B", "C"},
    {"A", "B", "C"}
]

from itertools import combinations

items = sorted(set().union(*transactions))

def support(itemset):
    return sum(1 for t in transactions
               if itemset.issubset(t))

# Candidate 1-itemsets
C1 = [{i} for i in items]

print("Candidate 1-itemsets:")
for c in C1:
    print(c, support(c))

# Candidate 2-itemsets
C2 = [set(c) for c in combinations(items,2)]

print("\nCandidate 2-itemsets:")
for c in C2:
    print(c, support(c))

print("\nFrequent Itemsets:")
for c in C1 + C2:
    if support(c) >= 2:
        print(c)

```

6. ECLAT Algorithm

```

transactions = [
    {"A", "B"},
    {"A", "C"},
    {"B", "C"}
]

min_sup = 2

tid = {}

for i, t in enumerate(transactions):

```

```

    for item in t:
        if item not in tid:
            tid[item] = set()

        tid[item].add(i)

print("TID Lists:", tid)

AB = tid["A"] & tid["B"]

print("Support of (A,B):", len(AB))

if len(AB) >= min_sup:
    print("(A,B) is Frequent")

```

7. Bagging

```

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.metrics import accuracy_score

X, y = make_classification(
    n_samples=400,
    n_features=4,
    random_state=0
)

Xtr, Xte, ytr, yte = train_test_split(
    X, y,
    test_size=0.3,
    random_state=0
)

bag = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=10,
    bootstrap=True,
    random_state=0
)

bag.fit(Xtr, ytr)

y_pred = bag.predict(Xte)

print("Accuracy:",
      accuracy_score(yte, y_pred))

```

10. Boosting

```

from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

```

```

ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=50,
    learning_rate=1.0,
    random_state=0
)

ada.fit(Xtr, ytr)

y_pred = ada.predict(Xte)

print("Boosting Accuracy:",
      accuracy_score(yte, y_pred))

```

11. Agent-Environment Interaction

```

import random

states = [0,1,2,3]
actions = ["Left", "Right"]

state = 0

for step in range(10):

    action = random.choice(actions)

    if action == "Right":
        state = min(state + 1, 3)
    else:
        state = max(state - 1, 0)

    reward = 1 if state == 3 else 0

    print("Step:", step+1,
          "Action:", action,
          "State:", state,
          "Reward:", reward)

```

12. Exploration vs Exploitation

```

import numpy as np

epsilon = 0.2

exploration = 0
exploitation = 0

for i in range(20):

    if np.random.rand() < epsilon:

        action = "Explore"
        exploration += 1

    else:

```

```

        action = "Exploit"
        exploitation += 1

    print("Iteration:", i+1,
          "Action:", action)

print("\nExploration:", exploration)
print("Exploitation:", exploitation)

```

13. Policy Representation

```

states = [0,1,2]

# Deterministic policy
policy_det = {
    0: "Left",
    1: "Right",
    2: "Right"
}

print("Deterministic Policy:")
print(policy_det)

# Stochastic policy
policy_sto = {
    0: {"Left":0.7, "Right":0.3},
    1: {"Left":0.4, "Right":0.6},
    2: {"Left":0.2, "Right":0.8}
}

print("\nStochastic Policy:")
print(policy_sto)

```

14. Markov Chain

```

import numpy as np

states = ["Sunny", "Rainy"]

P = np.array([
    [0.7, 0.3],
    [0.4, 0.6]
])

state = 0

count = [0, 0]

for day in range(10):

    print("Day", day+1, ":", states[state])

    count[state] += 1

    state = np.random.choice([0,1], p=P[state])

```

```
print("\nSunny Count:", count[0])
print("Rainy Count:", count[1])
```

15. Markov Reward Process

```
gamma = 0.9

R = {
    "S1":5,
    "S2":2
}

P = {
    "S1":{"S1":0.6,"S2":0.4},
    "S2":{"S1":0.3,"S2":0.7}
}

V1 = R["S1"] + gamma * (
    0.6 * R["S1"] +
    0.4 * R["S2"]
)

V2 = R["S2"] + gamma * (
    0.3 * R["S1"] +
    0.7 * R["S2"]
)

print("Value of S1:", round(V1,2))
print("Value of S2:", round(V2,2))
```

16. Policy Evaluation

```
import numpy as np

V = np.zeros(3)

gamma = 0.9

for i in range(10):

    V[0] = 1 + gamma * V[1]
    V[1] = 2 + gamma * V[2]
    V[2] = 3

    print("Iteration", i+1, ":", V)
```

17. Policy Improvement

```
import numpy as np

V = [5,8,10]

policy = []

for v in V:
```

```
    if v > 7:
        policy.append("Right")
    else:
        policy.append("Left")

print("Improved Policy:", policy)
```

18. Value Iteration

```
import numpy as np

V = np.zeros(3)

gamma = 0.9

for i in range(10):

    V[0] = max(1 + gamma*V[1],
               2 + gamma*V[2])

    V[1] = max(2 + gamma*V[2],
               1 + gamma*V[0])

    V[2] = 3

print("Optimal Values:", V)
```

19. Monte Carlo Method

```
episodes = [
    [1,2,3],
    [2,3],
    [1,1,2],
    [3,2],
    [2,2,1]
]

returns = []

for ep in episodes:

    G = sum(ep)

    returns.append(G)

    print("Episode Return:", G)

print("\nAverage Return:",
      sum(returns)/len(returns))
```

20. Q-Learning

```
import numpy as np

states = [0,1,2]
actions = [0,1]
```



```
Q = np.zeros((3,2))

alpha = 0.1
gamma = 0.9

for ep in range(20):

    state = np.random.choice(states)

    action = np.random.choice(actions)

    next_state = np.random.choice(states)

    reward = np.random.randint(0,5)

    Q[state, action] += alpha * (
        reward +
        gamma * np.max(Q[next_state]) -
        Q[state, action]
    )

print("Final Q-Table:\n")
print(Q)
```